UNIVERSITY OF THE PHILIPPINES - OPEN UNIVERSITY

CMSC 208: SOFTWARE ENGINEERING

FINAL EXAM

(2ND SEMESTER 2025-2026)

VERACISM AND SHOPEE.COM BLACK BOX TESTING REPORT

Members

Paolo Escotido
Jefferson Legaspi
Renald Cruz

PART A. PROPOSED COMPUTERIZED SYSTEM

VERACISM (Verifiable Academic Records for ISM)

I. Introduction (Company Background)

International School Manila handles a large volume of academic and administrative reports that must remain authentic, traceable, and resistant to tampering. The uploaded proposal and Software Requirements Specification describe a recurring problem in the current process: reports are generated by existing school applications and stored through standard APIs, but the institution lacks immutable version control, durable audit trails, and independent proof that a released file has not been changed after upload. As a result, verification requests still rely heavily on institutional trust, staff effort, and manual checking.

The proposed solution is VERACISM, or Verifiable Academic Records for ISM. It is a permanent, decentralized storage and verification service for finalized reports. Each file is scanned, validated, hashed, assigned a Content Identifier (CID), and anchored to decentralized storage through Lighthouse on IPFS with Filecoin-backed deals. Metadata and audit events remain under ISM control through MSSQL and role-based access controls. The result is a system that supports internal governance while also allowing external verifiers to confirm integrity through CID or QR-based checks.

II. Planning

A. Statement of the Problem

- Report files can be modified or replaced without a durable, append-only audit trail.
- There is no cryptographic binding between the stored file and the database record.
- Verification by external schools, auditors, or universities depends on manual email exchanges and staff confirmation.
- Multiple copies of the same report may exist across systems, increasing the risk of confusion and delays.
- The current process does not provide public, independent proof that a released file is unchanged.

B. Solution Strategy

The recommended strategy is to implement VERACISM as a hybrid solution. ISM retains control over identity, metadata, audit logs, and operational policy, while file integrity and long-term durability are strengthened through decentralized storage. Existing ISM systems upload finalized reports to a secure API. The system scans files through ClamAV and YARA, validates MIME type and size, computes a SHA-256 hash, stores the file through Lighthouse, records the CID, and writes the metadata and audit trail to MSSQL. A verification portal then allows authorized internal users and external verifiers to confirm report authenticity through CID or QR lookups.

The strategy addresses hardware, software, and personnel requirements as shown below.

Resource Area	Item	Purpose	Priority
Hardware	Application server / VM	Hosts the .NET API and web front end	High
Hardware	Database server / managed SQL instance	Stores tenant metadata, reports index, audit logs, and verification history	High
Software	Lighthouse + IPFS/Filecoin access	Provides content-addressed storage and durability	High
Personnel	Project team	Business analyst, developers, QA/security, DBA/DevOps, and tenant representatives	High

C. Project Schedule

A sixteen-week delivery plan is appropriate because the proposed system has clear functional modules, security controls, and an external verification workflow. The schedule below adapts the development sequence in the proposal into a concise academic project plan.

Week	Stage	Main Output	Main Output
1-2	Planning and requirements	Interview findings, scope, risk register, draft backlog	PM / BA
3-4	Analysis	DFD, narratives, data dictionary, acceptance criteria	BA / Architect
5-7	Design	Architecture, database design, interface sketches, security controls	Architect / UI
8-11	Implementation setup	API design, module structure, DevSecOps pipeline, integration guides	Dev team
12-13	Integration testing	End-to-end upload, scan, CID, and verification workflow	QA / Dev
14	Performance and security test	Load, VAPT preparation, remediation list	QA / Security
15	UAT and revisions	Tenant review, defect fixes, sign-off package	Stakeholders
16	Deployment readiness	Runbooks, cutover checklist, final release package	DevOps / PM

D. Cost Estimation

The cost estimate below assumes a short implementation cycle using existing ISM infrastructure where possible. Costs are shown in Philippine pesos and include labor, core infrastructure, security setup, and contingency.

Cost Item	Basis	Amount (PHP)	Remarks
Project management and	3 months	240,000	Planning, interviews,
Back-end development	4 months	280,000	Core upload, hashing, CID, audit
Front-end development	4 months	240,000	Internal console and public portal
QA and security testing	2 months	100,000	Functional, performance, and
DBA / DevOps support	2 months	120,000	MSSQL, deployment,
Infrastructure and tools	Service setup	85,000	Server allocation, storage,
Contingency	Approx. 5%	60,000	Covers minor overruns and fixes

Estimated Total: PHP 1,125,000

Software Engineering Tools for Each Development Stage

Stage	Primary Tool	Use in Project	Output
Planning	Google Docs / MS Word	Interview notes, problem	Project charter
Analysis	Draw.io or Lucidchart	DFD creation, process mapping,	Analysis models
Design	Figma, Draw.io, dbdiagram.io	Interface wireframes,	Design package
Implementation	Visual Studio / VS Code, Git	API, front-end modules,	Build-ready codebase
Testing	Postman, JMeter, OWASP ZAP /	Functional, API, load, and security	Test reports

III. Requirement Analysis

A. Data Flow Diagram

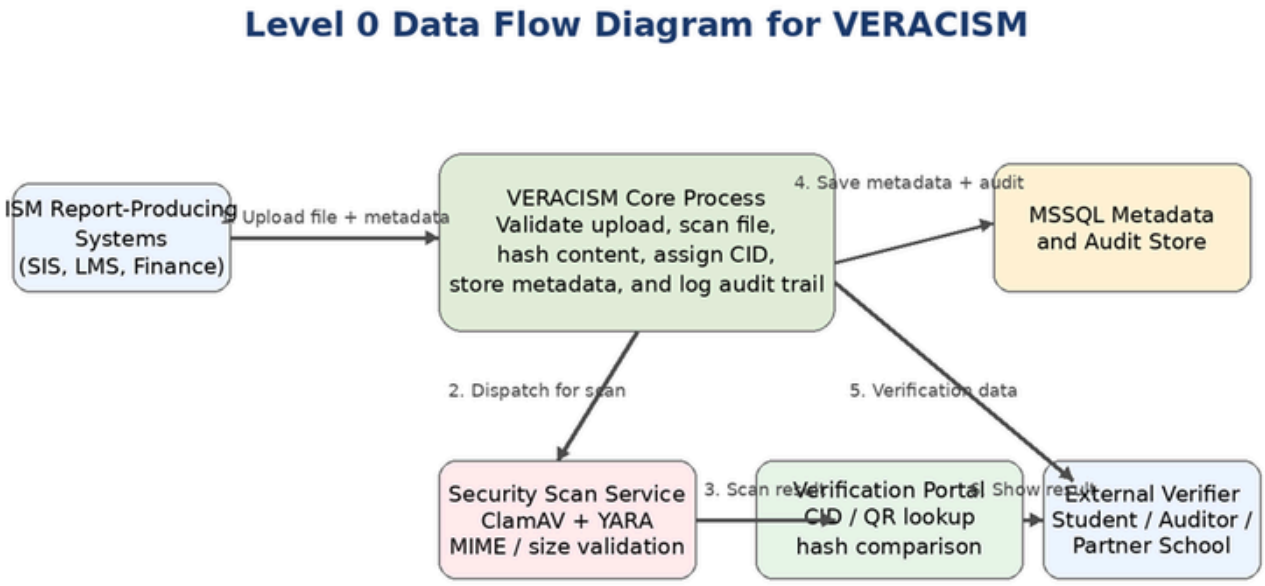

Figure 1. Level 0 data flow diagram of the proposed VERACISM workflow.

The diagram shows the central data movement. Existing ISM systems submit a report file and metadata. VERACISM validates and scans the upload, stores report metadata and audit events in MSSQL, and exposes verification data to the public-facing portal. External verifiers only see the result of the CID or QR check and do not need direct credentials to confirm report integrity.

B. Narrative Descriptions

ID	Process	Narrative Description
P1	Secure upload	Authorized systems send finalized reports and metadata through the upload API. The system rejects invalid, oversized, or unauthorized submissions.
P2	Security validation	Each accepted file is scanned through ClamAV and YARA, then checked for allowed type and size before it proceeds.
P3	Hashing and storage	The system computes a SHA-256 hash, requests decentralized storage through Lighthouse, receives the CID, and records the proof details.
P4	Metadata and audit logging	MSSQL stores the report record, tenant metadata, verification details, and append-only audit events for traceability and compliance.

C. Data Dictionary

Data Element	Type	Source	Description
tenant_id	Integer	Tenant table	Unique organization
student_id	Integer	Student table	References the student linked to
report_id	Integer	Report table	Primary key for each finalized
filename	Varchar	Upload payload	Original report file name supplied by
mime_type	Varchar	Validation layer	Recorded content type after file
sha256	Hash	Hash service	Cryptographic digest used for
cid	Varchar	Lighthouse / IPFS	Content Identifier returned after
audit_id	Integer	AuditEvent table	Unique ID for an append-only

IV. Software Design

A. Data Design

The core data model is tenant-aware. Tenant, AppUser, Role, UserRole, Student, Report, ScanResult, VerificationLog, and AuditEvent are the essential entities. Report sits at the center because it links a student, the source system, the computed hash, the CID, storage references, and the submission timestamp. ScanResult stores one or more malware or validation outcomes for a report. VerificationLog records every CID or QR-based check, including whether the observed hash matches the stored hash. AuditEvent stores append-only operational evidence for security, traceability, and compliance.

Entity	Key Attributes	Relationship Summary
Tenant	tenant_id, code, name	Owns users, API clients, students, reports, and audit events.
AppUser	user_id, upn, display_name	Belongs to one tenant and can hold multiple roles.
Student	student_id, ext_ref, full_name	Belongs to one tenant and can have many reports.
Report	report_id, sha256, cid, status	Belongs to one tenant and one student; has scan and verification history.
ScanResult	scan_id, engine, result	Linked to a single report and stores malware or rule outcomes.

B. Architectural Design

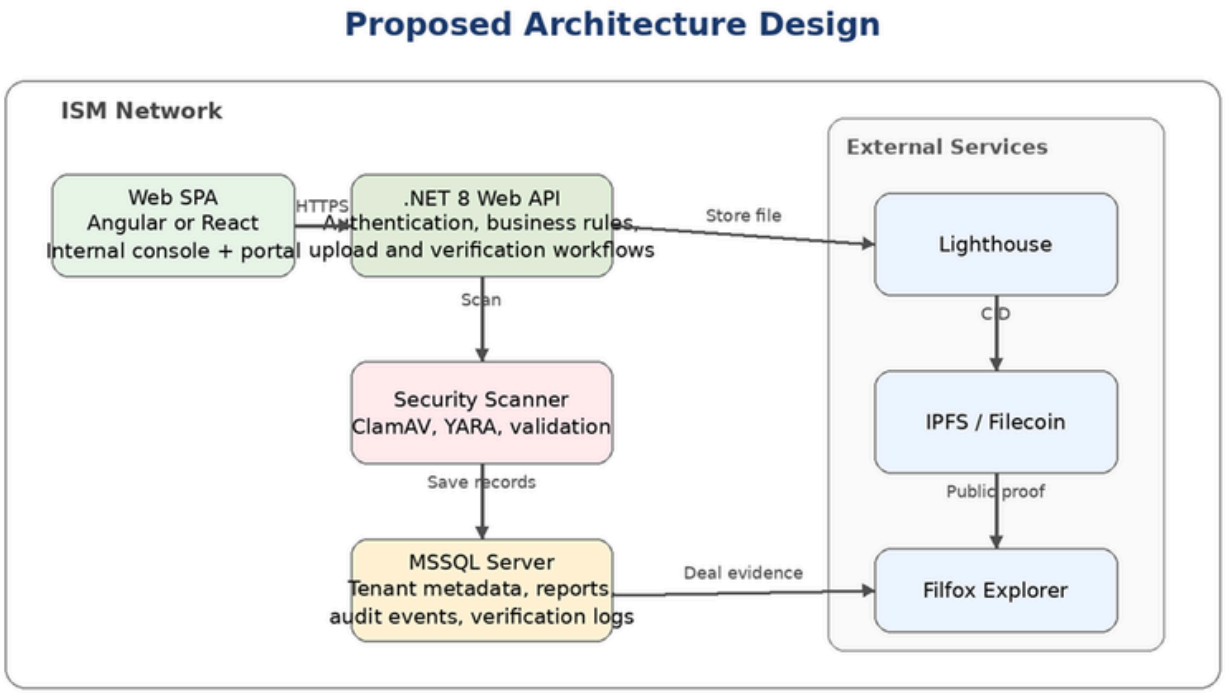

Figure 2. High-level architecture design of the proposed system.

The architecture follows a layered model. The web front end is a single-page application. The .NET 8 API enforces authentication, business rules, and workflow control. A dedicated scan service performs malware and file-type checks. MSSQL stores operational records under ISM control. Lighthouse and IPFS/Filecoin provide decentralized storage and public evidence. This separation improves maintainability, security hardening, and operational visibility.

C. Procedural Design

Procedure	Main Steps
Upload procedure	Authenticate request -> validate metadata -> scan file -> compute SHA-256 -> request Lighthouse storage -> save CID, metadata, and audit event -> return status and QR/CID.
Verification procedure	Receive CID or QR -> retrieve report metadata -> fetch file by CID -> recompute hash -> compare values -> display match, mismatch, or not found result.
Monitoring procedure	Run scheduled health checks -> inspect CID availability and latency -> log anomalies -> send alert to operators.
Key management procedure	Restrict updates to admins -> store secrets in managed vault -> log all key rotations and configuration changes.
Incident response	Review audit trail -> isolate affected tenant or service account -> fix root cause -> retest -> document closure.

D. Interface Design

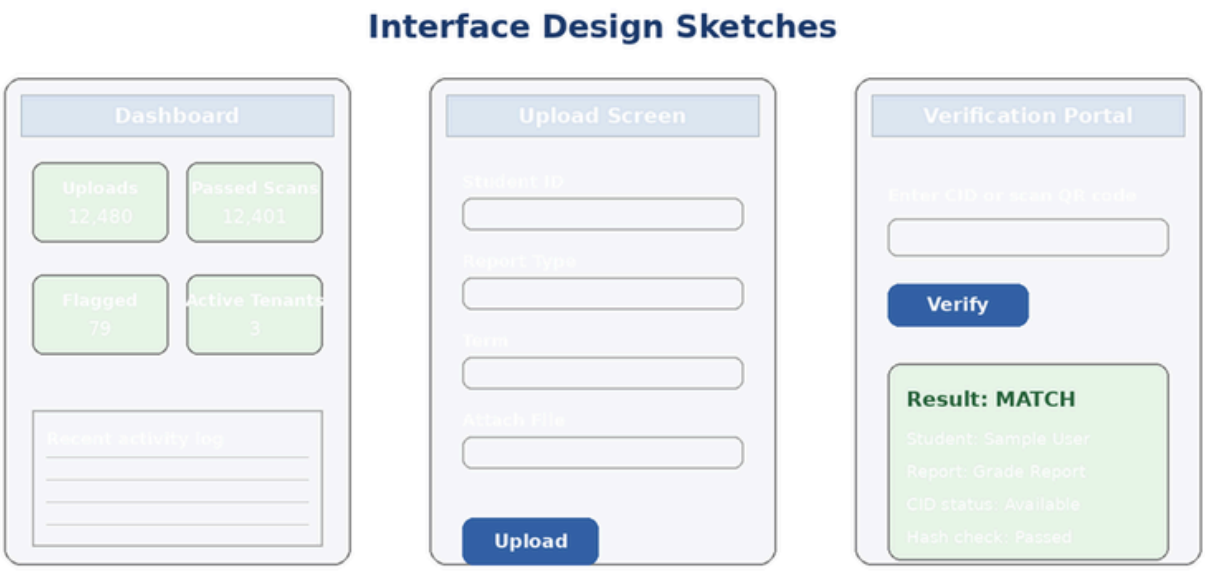

Figure 3. Suggested interface sketches for dashboard, upload, and verification pages

The interface design emphasizes clarity and task completion. The dashboard surfaces upload volume, scan results, flagged items, and tenant activity. The upload screen minimizes error by using structured fields and a clear action button. The verification portal is intentionally simple so that students, auditors, and partner schools can enter a CID or scan a QR code and immediately understand the result. Result messages should use plain language such as Match, Mismatch, or Not Found.

V. Software Implementation

A. Programming Language

C# is the most suitable programming language for the back-end because the proposal and SRS already assume a .NET 8 Web API. It is appropriate for secure enterprise development, integrates well with MSSQL, supports modern API patterns, and fits ISM operational standards. TypeScript is recommended for the front end because it improves maintainability and reduces runtime errors in a large interface codebase.

B. Framework

ASP.NET Core 8 is recommended for the API and business layer. It supports RESTful services, authentication middleware, dependency injection, structured logging, and secure deployment patterns. For the front end, Angular or React with TypeScript is appropriate. React offers flexibility and component reuse, while Angular provides stronger built-in structure. For this project, React with TypeScript would be a practical option because the interface is modular and can be split into dashboard, upload, configuration, and verification modules.

C. Database Type

Microsoft SQL Server is the recommended database type. It supports relational integrity, transaction management, indexing, role-aware access, and reporting. These capabilities are essential for report metadata, verification logs, scan results, and immutable audit events. MSSQL also aligns with the design materials already present in the proposal and SRS.

Implementation Justification

The selected stack is coherent. ASP.NET Core and MSSQL align with ISM standards described in the supporting documents. TypeScript on the client side supports scalable UI development. Lighthouse and IPFS/Filecoin are treated as external storage services rather than the system of record, which keeps sensitive metadata and policy control within the institution while still providing independent proof of integrity.

PART B. SYSTEM TESTING AND MAINTENANCE

Chosen Application: Shopee.com

I. Introduction

Shopee.com is a widely used online shopping platform that lets users browse products, compare offers, read ratings and reviews, place items in a cart, choose payment and delivery options, and track orders through a web interface. Its public features include home-page merchandising, product search, filters, product detail pages, vouchers, checkout, and order-status views. Because the internal code and infrastructure are not visible to the tester, Shopee.com is an appropriate subject for black box testing. The platform can still be examined through user-facing inputs, outputs, response times, and behavior under normal and stressful conditions..

II. System Testing

A. Functional Test

TC	Test Scenario	Expected Result	Observation
F1	Search for a common product keyword	Relevant product listings should appear with images, prices, and shop information	The platform presents structured search results that help users compare options quickly.
F2	Open a product page from search results	Product page should load with title, price, variations, ratings, shipping details, and purchase actions	The product detail page groups the main purchase information into a clear buyer flow.
F3	Select a variation and add an item to cart	Chosen variation should be accepted and the cart count should update correctly	Variation selection and cart feedback are core visible functions in the buying flow.

F4	Apply a voucher or promo during checkout	Eligible discount should appear correctly before order placement	Checkout behavior should clearly show whether a promotion is accepted or rejected.
F5	Choose a payment method and proceed to order review	System should move to the next checkout step without losing cart data	The checkout sequence is designed to keep shipping, payment, and order summary information visible.
F6	Open the order page after purchase or from account history	User should see order status and tracking-related updates when available	Order-status visibility is an important user-facing trust feature of the platform.

B. Performance Test

PT	Scenario	Expected Result	Observation
P1	Load the Shopee.com home page on a supported browser	Home page should render without major layout breakage	Shopee.com relies on modern browser behavior and dynamic page elements for merchandising and navigation.
P2	Search for a common item on a stable connection	Results page should load quickly enough for normal browsing	Search responsiveness is central to the shopping experience because users compare many items in sequence.
P3	Open a product page with images, ratings, and seller details	Product information should load with acceptable delay and usable layout	A product page carries more interface elements than a simple listing page, so it is a good visible performance check.
P4	Move from cart to checkout on an older or unsupported browser	System should still guide the user clearly or warn about compatibility issues	Shopee help guidance indicates that older browser versions can cause loading or processing problems on the website.

C. Stress Test

ST	Scenario	Expected Result	Observation
S1	Open multiple product pages in separate tabs	Core browsing should remain usable, though individual pages may load more slowly	This checks whether the client-side shopping flow degrades gracefully under heavier user demand.
S2	Repeat searches and filter changes in quick succession	The system should continue returning results without major visible errors	Search stability matters because shopping behavior often involves rapid comparison across many listings.
S3	Add and remove several items from the cart repeatedly	Cart contents and totals should stay consistent instead of becoming inaccurate	Cart consistency is a visible stress point because it combines state changes and price calculations.
S4	Reduce connection quality while loading checkout or order pages	The platform should retry, partially load, or show a clear error instead of failing silently	This tests resilience when the user experiences unstable connectivity during a critical transaction stage.

III. Maintenance

A. Version History

Period	Enhancement / Bug Fix Theme	Notes
Recent public releases	Bug fixes and performance enhancements	The public Shopee app listing repeatedly reports bug fixes and performance improvements, which suggests ongoing maintenance across the wider Shopee platform.
Current platform features	Search, vouchers, reviews, logistics, and payment support	Public product descriptions continue to highlight buyer-facing features such as ratings, reviews, secure payment handling, and order tracking.
Current website support guidance	Browser compatibility and troubleshooting	Shopee help documentation notes that older browser versions may not support newer website features and recommends updates, cache clearing, or switching browsers when issues occur.
Ongoing marketplace evolution	Incremental feature refinement instead of radical redesign	Public-facing evidence suggests that Shopee improves reliability, compatibility, and shopping convenience continuously while keeping the main buying flow familiar to users.

B. Observations

- Shopee.com appears to be maintained through frequent incremental updates, campaign refreshes, and compatibility fixes rather than infrequent major redesigns.
- Public maintenance signals emphasize smoother browsing, stable checkout, stronger device and browser compatibility, and continuous bug and performance improvements.
- The platform's core buy-search-checkout-track flow remains stable, while surrounding features such as promotions, recommendations, and payment options continue to evolve.
- Shopee.com is suitable for black box testing because quality can be judged from visible behavior such as page loading, search accuracy, cart handling, checkout flow, and order-status feedback even without seeing the back-end logic.

Conclusion

This final exam shows how software engineering concepts apply both to a proposed information system and to a widely used live platform. In Part A, VERACISM addresses a real institutional problem through structured planning, analysis, design, and implementation decisions. In Part B, Shopee.com demonstrates how black box testing can evaluate user-facing quality through functional, performance, stress, and maintenance perspectives. Together, the two parts reflect the central software engineering goal of building and maintaining systems that are reliable, secure, useful, and verifiable.

References

Final exam brief; VERACISM proposal; VERACISM SRS; official Shopee Help Center buying guide; official Shopee Help Center website troubleshooting guide; and the public Shopee PH Google Play listing for current feature and maintenance notes.